Tutorial 3: Monte Carlo Simulations

Richard Lockhart: based on similar tutorials by Isaac Marchand

2026-09-24

Some extra requests for what you should hand in:

1) Your Crowdmark paper
2) A single .Rmd file for each group of 3 or 4 students
3) Please change the name from Tutorial3ForStudents.Rmd to something that includes the name of the student emailing the file to me.
4) Please make sure the file contains the names of all the students in the group.
5) Please make sure the file knits successfully.

Preparing the Workspace

Install, if necessary, and import the necessary packages.

Introduction

This tutorial asks you to do two small simulation studies. They serve two purposes. First you will examine what Q-Q plots look like when the underlying variables are truly iid. And second you will evaluate the null hypothesis behaviour of the runs test applied to residuals from a linear regression model.

You will simulate time series (mostly white noise!), fit a trend model, and compute standardized residuals which will then be checked for normality and stationarity visually or using formal tests.

In Monte Carlo simulation we use (pseudo) random number generators to mimic the behaviour of sampling from various population distributions. I am sure you have all used **runif** to generate observations with a uniform distribution or **rnorm** to generate normally distributed observations.

Part 1. Q-Q plots

I want you to:

a) generate a sample, x, of size n = 72 (this is the number of observations in the wages data set from the previous tutorial) from the standard normal distribution.

b) standardize the sample by computing z =(x-mean(x))/sd(x)

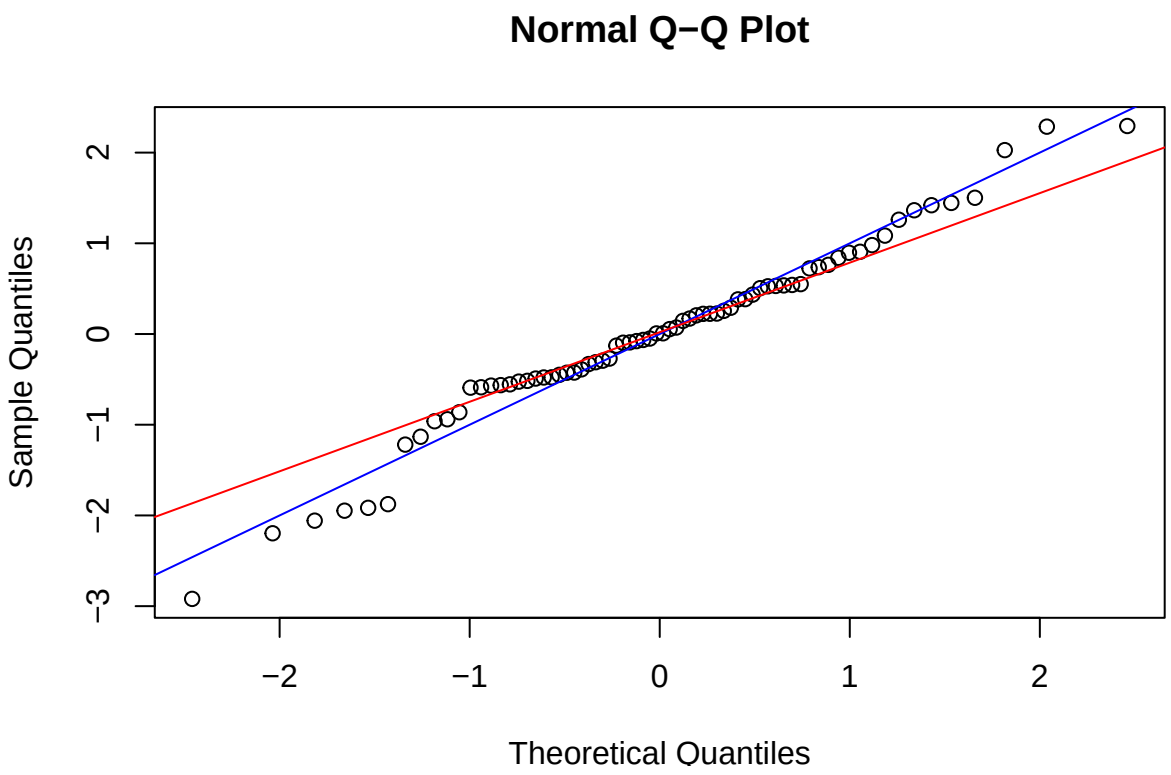
c) use **qqnorm** to make a Q-Q plot of the z values.

d) add a line with slope 1 and intercept 0 to the plot and a line using **qqline**.

e) examine the plot to see how far the plotted points are from the line.

Here is the code I used. The command set.seed makes the results the same every time you run the same code.

```r
set.seed(1943)
n = 72
x = rnorm(n)
z = (x-mean(x))/sd(x)
qqnorm(z)
abline(0,1,col='blue')
qqline(z,col='red')
```



This next bit of code makes 6 Q-Q plots for 6 samples of size n. It ensures that the 6 plots appear in a 3 by 2 matrix, that there is no space between the panels, that the x and y axes of all plots have the same scale.

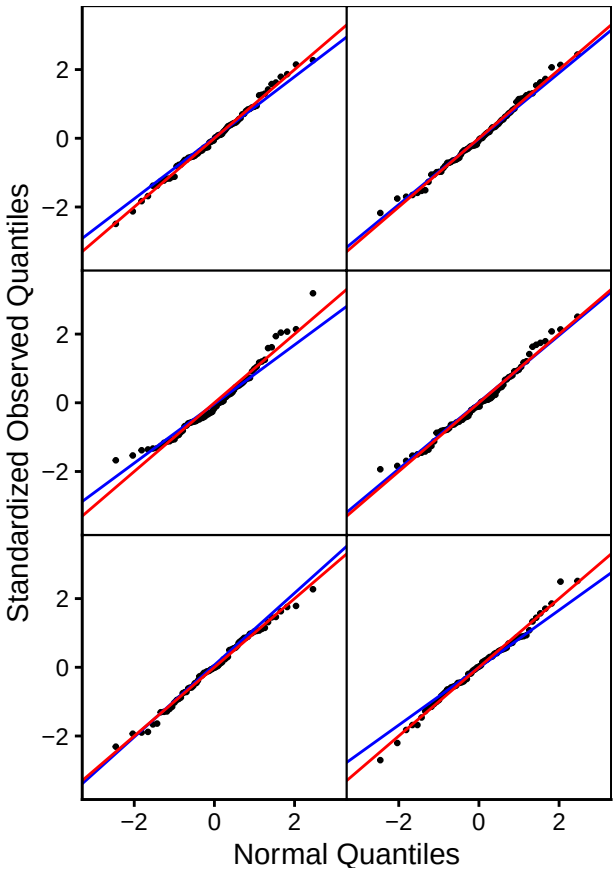
```r
# Generate 6 samples of size n and standardize each one
set.seed(123)

x <- matrix(rnorm(6 * n), nrow = n, ncol = 6)
x <- scale(x)

# Put the six samples into long format
d <- data.frame(
  value = as.vector(x),
  sample = factor(rep(1:6, each = n))
)

# QQ plots
ggplot(d, aes(sample = value)) +
  scale_x_continuous(limits = c(-3, 3)) +
  scale_y_continuous(limits = c(-3.5, 3.5)) +
  stat_qq(size = 0.5) +
  stat_qq_line(color = "blue") +
  geom_abline( intercept = 0, slope = 1,color="red") +
```

```
  labs(x = "Normal Quantiles", y = "Standardized Observed Quantiles")+
  facet_wrap(~ sample, nrow = 3, ncol = 2) +
  coord_fixed() +
  theme_classic() +
  theme(
    panel.spacing = unit(0, "pt"),
     panel.border = element_rect(
      color = "black",
      fill = NA,
      linewidth = 0.4
    ),
    aspect.ratio = 1,
    plot.margin = margin(0, 0, 0, 0),
    strip.background = element_blank(),
    strip.text = element_blank()
  )
```



I want you to run that last code chunk several times using the console and remembering to load the ggplot2 libary. Use set.seed only once so that the different sets of 6 graphs are not all the same.

What you are supposed to learn here: I want you to comment on which of the 2 lines does a better job of tracking the data overall. I also want you to just look through the graphs to get intuition about how variable they are when the data are exactly normal (but standardized). I would also like you to try this again **without** the line of code **x <- scale(x)** which standardizes each column of x. Again comment on which line seems to work better.

Part 2: the runs test

In this section we are going to generate samples which are white noise, apply the runs test to each sample, save the results and examine of the p-values. We will use a fairly large number of samples, all the same size – 72 again.

Then we will repeat this with data generated from a linear regression model.

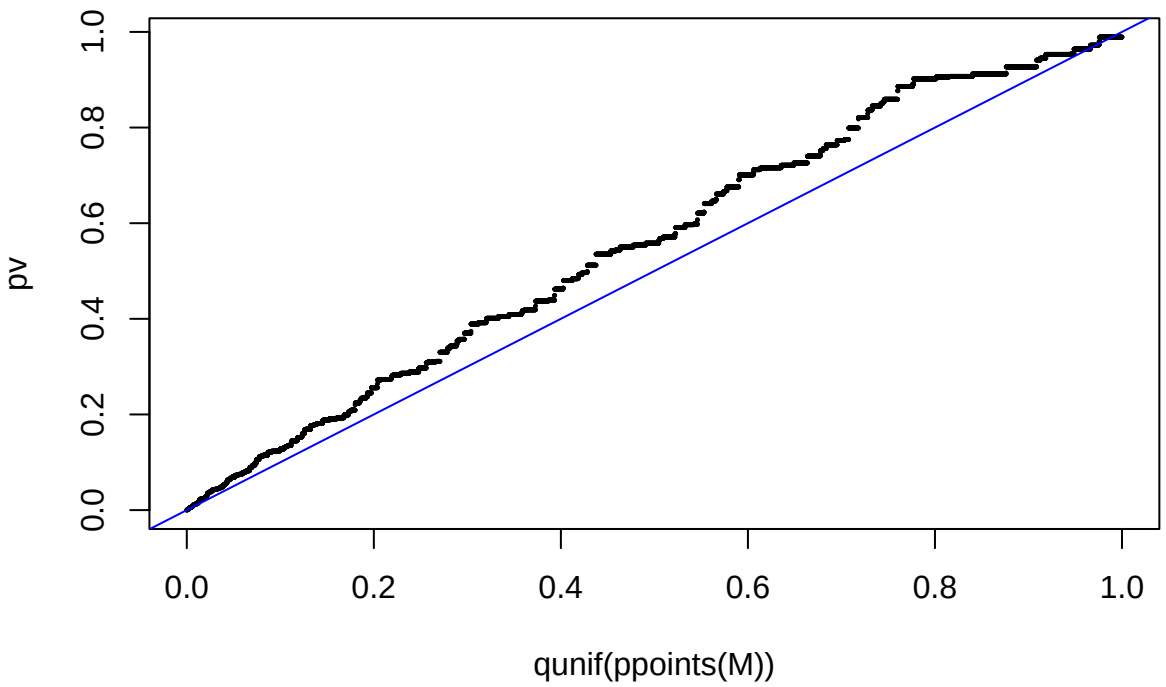
What should happen? In each case the null hypothesis will be true: exactly when applied to white noise but only approximately when we use regression residuals.

Here is some basic code to get you started.

```
# white noise
M = 10000
n = 72
set.seed(1943)
e = matrix(rnorm(n*M),nrow = n) # while noise errors
pv = apply(e,2,function(x){runs(x)$pvalue})
length(unique(pv))
```

```
## [1] 273
```

```
qqplot(qunif(ppoints(M)),pv,pch=19,cex=0.2)
abline(0,1,col='blue')
```



Please explain what the code line **pv = ...** and the line right after that do. The function **ppoints** computes M numbers evenly spaced between 1/20000 and 1 - 1/20000.

Find out how many of the pvalues are larger than 0.05. This number divided by 10000 is a Monte Carlo estimate of the probability of making a type one error. The null hypothesis is definitely correct here.

```
## [1] 389
```

A hypothesis test is conservative if the fraction of pvalues smaller than 0.05 is less that 0.05 – it is cautious about rejecting. It is well calibrated if that fraction is close to 0.05. And it is liberal if the fraction is more than 0.05.

You should be able to get a confidence interval for the true level of this test for n=72 by using a confidence interval for a binomial proportion. Is 0.05 inside that confidence interval?

Part 3: residuals

I worry that you will run out of time for this so here is code.

```
# white noise
M = 10000
n = 72

desmat = cbind(rep(1,n),qnorm(ppoints(n))) # Design matrix
hatmat = desmat%*% solve(t(desmat)%*%desmat,t(desmat))
id.minus.hatmat = diag(n)-hatmat
sum(diag(hatmat))
```

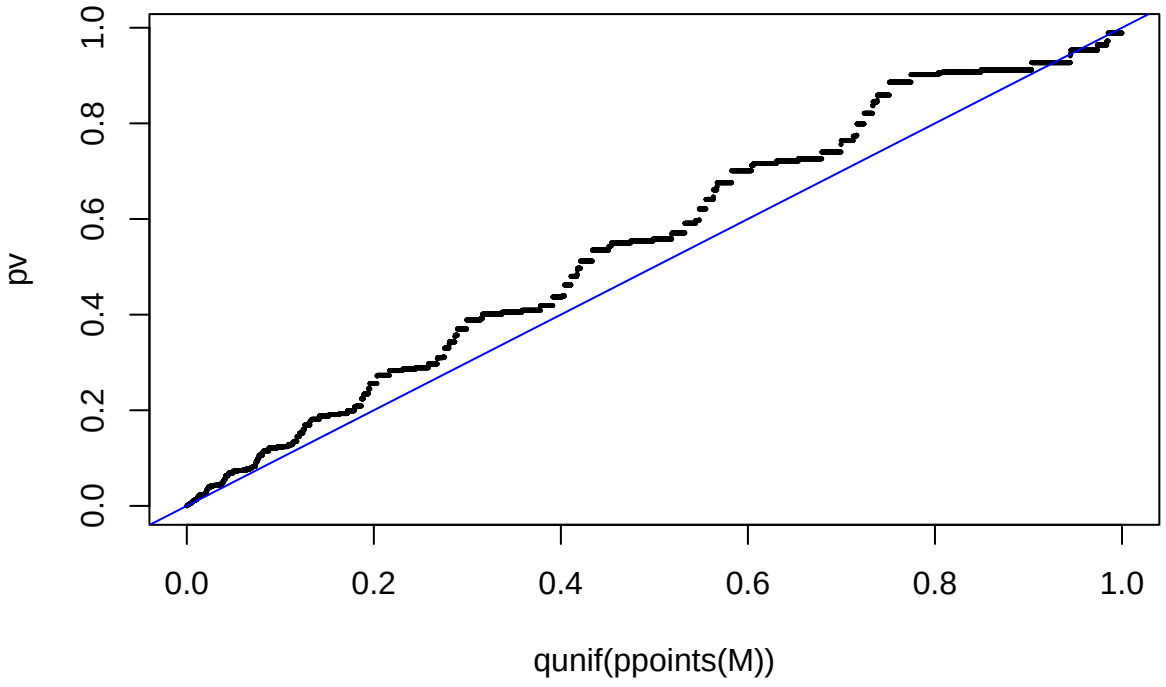
```
## [1] 2
```

```
set.seed(1943)
e = matrix(rnorm(n*M),nrow = n) # while noise errors
res = id.minus.hatmat %*% e
pv = apply(res,2,function(x){runs(x)$pvalue})

length(unique(pv))
```

```
## [1] 190
```

```
qqplot(qunif(ppoints(M)),pv,pch=19,cex=0.2)
abline(0,1,col='blue')
```



Here is my count of the number of pvalues less than or equal to 0.05. It then prints a summary of the 10000 pvalues.

```
## [1] 389
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 0.0000177 0.2890000 0.5580000 0.5541365 0.8590000 0.9890000
```

If you have time find out if the test is liberal, well calibrated, or conservative. This time the residuals are not independent so the null hypothesis is not exactly true. But they are not very dependent because we are only estimating 2 parameters and there are no really influential observations.